

```

# -*- coding: utf-8 -*-
# =====
# AI 波段預測系統 Web App (Streamlit)
# Tab 1: 每日信號 (V2.1) | Tab 2: 損益回測 (V3)
# =====

import numpy as np
import pandas as pd
import yfinance as yf
import streamlit as st
import matplotlib.pyplot as plt
from xgboost import XGBClassifier
import warnings
warnings.filterwarnings("ignore")

HORIZON = 10
TARGET_RET = 0.02
ATR_MULT = 2.0
N_FOLDS = 5
EDGE_MIN = 0.05

st.set_page_config(page_title="AI Swing System", page_icon="📈", layout="centered")
st.title("AI 波段預測系統 (SPY)")

# ----- 側欄參數 -----
st.sidebar.header("系統參數")
PROB_THRESHOLD = st.sidebar.slider("買入概率閾值", 0.50, 0.80, 0.60, 0.01)
if st.sidebar.button("強制刷新數據"):
    st.cache_data.clear()

# ----- 數據與特徵 -----
@st.cache_data(ttl=3600, show_spinner="正在下載市場數據...")
def load_data():
    data = yf.download(["SPY", "QQQ", "^VIX"], start="2018-01-01", progress=False)
    df = pd.DataFrame(index=data.index)
    df["SPY_Open"] = data["Open"]["SPY"]
    df["SPY_Close"] = data["Close"]["SPY"]
    df["SPY_High"] = data["High"]["SPY"]
    df["SPY_Low"] = data["Low"]["SPY"]
    df["QQQ_Close"] = data["Close"]["QQQ"]
    df["VIX_Close"] = data["Close"]["^VIX"]

    df["VIX_MA10_Ratio"] = df["VIX_Close"] / df["VIX_Close"].rolling(10).mean()
    df["SPY_MA200_Ratio"] = df["SPY_Close"] / df["SPY_Close"].rolling(200).mean()
    df["QQQ_SPY_Ratio"] = df["QQQ_Close"] / df["SPY_Close"]

```

```

df["Breadth_Momentum"] = df["QQQ_SPY_Ratio"].pct_change(20)

delta = df["SPY_Close"].diff()
gain = delta.where(delta > 0, 0).rolling(14).mean()
loss = (-delta.where(delta < 0, 0)).rolling(14).mean()
df["SPY_RSI"] = 100 - (100 / (1 + gain / loss))

prev_close = df["SPY_Close"].shift(1)
tr = pd.concat([
    df["SPY_High"] - df["SPY_Low"],
    (df["SPY_High"] - prev_close).abs(),
    (df["SPY_Low"] - prev_close).abs()
], axis=1).max(axis=1)
df["ATR14"] = tr.rolling(14).mean()

df["Future_Return"] = df["SPY_Close"].shift(-HORIZON) / df["SPY_Close"] - 1
df["Target"] = (df["Future_Return"] > TARGET_RET).astype(int)
return df

FEATURES = ["VIX_MA10_Ratio", "SPY_MA200_Ratio", "Breadth_Momentum", "SPY_RSI"]

def make_model():
    return XGBClassifier(n_estimators=150, max_depth=4,
                        learning_rate=0.03, random_state=42,
                        eval_metric="logloss")

@st.cache_data(ttl=3600, show_spinner="正在執行 Walk-Forward 驗證...")
def run_walk_forward(df):
    feat_df = df.dropna(subset=FEATURES + ["ATR14"])
    train_df = feat_df.dropna(subset=["Future_Return"]).copy()
    n = len(train_df)
    test_size = n // (N_FOLDS + 1)
    prob_series = pd.Series(dtype=float)
    for i in range(N_FOLDS):
        test_start = n - (N_FOLDS - i) * test_size
        test_end = test_start + test_size
        train_end = test_start - HORIZON
        m = make_model()
        m.fit(train_df[FEATURES].iloc[:train_end], train_df["Target"].iloc[:train_end])
        p = m.predict_proba(train_df[FEATURES].iloc[test_start:test_end])[:, 1]
        prob_series = pd.concat([prob_series,
                                pd.Series(p, index=train_df.index[test_start:test_end])])
    final_model = make_model()
    final_model.fit(train_df[FEATURES], train_df["Target"])
    latest_row = feat_df.iloc[[-1]]
    latest_prob = float(final_model.predict_proba(latest_row[FEATURES])[0][1])
    return prob_series, train_df, feat_df, latest_prob

```

```

df = load_data()
prob_series, train_df, feat_df, latest_prob = run_walk_forward(df)

# OOS 驗證統計
base_rate = train_df["Target"].mean()
mask = prob_series > PROB_THRESHOLD
n_signals = int(mask.sum())
oos_winrate = train_df.loc[prob_series.index[mask], "Target"].mean() if n_signals
edge = oos_winrate - base_rate if n_signals else np.nan
model_valid = (n_signals >= 20) and (edge >= EDGE_MIN)

latest_date = feat_df.index[-1].strftime("%Y-%m-%d")
latest_price = float(feat_df["SPY_Close"].iloc[-1])
latest_atr = float(feat_df["ATR14"].iloc[-1])

tab1, tab2 = st.tabs(["📊 每日信號", "✍️ 損益回測"])

# ===== Tab 1: 每日信號 =====
with tab1:
    st.caption(f"數據截至: {latest_date}")
    c1, c2, c3 = st.columns(3)
    c1.metric("SPY 收盤", f"${latest_price:.2f}")
    c2.metric("ATR(14)", f"${latest_atr:.2f}")
    c3.metric("10日漲>2% 概率", f"{latest_prob:.1%}")

    st.divider()
    st.subheader("樣本外驗證 (Walk-Forward)")
    v1, v2, v3 = st.columns(3)
    v1.metric("歷史基準率", f"{base_rate:.1%}")
    v2.metric("信號勝率 (OOS)", f"{oos_winrate:.1%}" if n_signals else "N/A")
    v3.metric("統計邊際", f"{edge:+.1%}" if n_signals else "N/A")
    if model_valid:
        st.success(f"模型有效性檢驗: PASS (信號 {n_signals} 次)")
    else:
        st.error(f"模型有效性檢驗: FAIL (信號 {n_signals} 次) - 禁止採用任何信號")

    st.divider()
    st.subheader("系統指令")
    if latest_prob > PROB_THRESHOLD and model_valid:
        stop_price = latest_price - ATR_MULT * latest_atr
        target_price = latest_price * (1 + TARGET_RET)
        st.success("🟢 BUY - 10 日波段做多")
        b1, b2 = st.columns(2)
        b1.metric("目標價 (+2%)", f"${target_price:.2f}")
        b2.metric(f"ATR 止損 (-{ATR_MULT*latest_atr/latest_price:.1%})", f"${stop_
st.caption("執行規則: 次日開盤進場 | 觸及目標/止損/滿10個交易日 先到先平倉")

```

```

elif latest_prob > PROB_THRESHOLD:
    st.warning("🟡 概率達標但模型未通過有效性檢驗 - 禁止進場")
else:
    st.info("🟢 NEUTRAL - 觀望, 不進場")

# ===== Tab 2: 損益回測 =====
with tab2:
    all_dates = df.index
    trades = []
    in_pos_until = -1
    for sig_date, p in prob_series.items():
        if p <= PROB_THRESHOLD:
            continue
        loc = all_dates.get_loc(sig_date)
        if loc <= in_pos_until or loc + 1 >= len(all_dates):
            continue
        entry_loc = loc + 1
        entry = float(df["SPY_Open"].iloc[entry_loc])
        atr = float(df["ATR14"].iloc[loc])
        stop = entry - ATR_MULT * atr
        target = entry * (1 + TARGET_RET)
        exit_ret, exit_loc, reason = None, None, None
        for d in range(entry_loc, min(entry_loc + HORIZON, len(all_dates))):
            if float(df["SPY_Low"].iloc[d]) <= stop:
                exit_ret, exit_loc, reason = stop / entry - 1, d, "STOP"
                break
            if float(df["SPY_High"].iloc[d]) >= target:
                exit_ret, exit_loc, reason = TARGET_RET, d, "TARGET"
                break
        if exit_ret is None:
            exit_loc = min(entry_loc + HORIZON - 1, len(all_dates) - 1)
            exit_ret = float(df["SPY_Close"].iloc[exit_loc]) / entry - 1
            reason = "TIME"
        in_pos_until = exit_loc
        trades.append({"信號日": sig_date.strftime("%Y-%m-%d"),
                      "報酬": exit_ret, "出場": reason})

    tdf = pd.DataFrame(trades)
    if len(tdf) == 0:
        st.info("當前閾值下沒有產生任何樣本外交易。")
    else:
        wins = tdf[tdf["報酬"] > 0]
        losses = tdf[tdf["報酬"] <= 0]
        equity = (1 + tdf["報酬"]).cumprod()
        mdd = ((equity - equity.cummax()) / equity.cummax()).min()

        m1, m2, m3 = st.columns(3)

```

```

m1.metric("交易筆數", len(tdf))
m2.metric("勝率", f"{len(wins)/len(tdf):.1%}")
m3.metric("期望值/筆", f"{tdf['報酬'].mean():+.3%}")
m4, m5, m6 = st.columns(3)
m4.metric("總回報", f"{(equity.iloc[-1]-1):+.1%}")
m5.metric("最大回撤", f"{mdd:.1%}")
m6.metric("平均盈/虧", f"{wins['報酬'].mean():+.2%} / {losses['報酬'].mean()}")
st.caption("出場分佈: " + str(tdf["出場"].value_counts().to_dict()))

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(equity.values)
ax.axhline(1.0, color="gray", lw=0.5)
ax.set_title("Equity Curve (OOS trades)")
ax.set_xlabel("Trade #")
ax.grid(alpha=0.3)
st.pyplot(fig)

with st.expander("查看全部交易紀錄"):
    show = tdf.copy()
    show["報酬"] = (show["報酬"] * 100).round(2).astype(str) + "%"
    st.dataframe(show, use_container_width=True)

st.divider()
st.caption("本系統僅供研究參考，不構成投資建議。每季須確認有效性檢驗為 PASS。")

```